Tutor System Architecture — Audited Repository Summary

Date: 2026-05-27

This document is an audited, in-depth summary of the Tutor System Architecture repository after a focused pass through the core runtime, UI, memory, and server layers. It is grounded in the current source tree and highlights the live data flows, key modules, and operational boundaries.

1) Scope of Review

Reviewed layers and entry points

- **Core runtime:** `src/App.tsx`, `src/store/index.ts`, `src/brain-runtime/*`
- **UI:** `src/views/*`, `src/components/*`
- **Memory:** `src/memory/*`
- **Server:** `server.ts`, `server/web-search.ts`

Audit goal: describe how the system runs end-to-end, what each layer owns, and how the user experience is powered by the data and runtime instrumentation.

2) Repository Topography (High-Level)

- **Frontend (React + Vite):** `src/`
  - Views and components render the PDF study surface, chat, analytics, revision, and admin diagnostics.
- **Server (Express):** `server.ts`
  - SSE chat, web search, TTS, Deepgram voice agent proxy, debug-run orchestration.
- **Memory + Learner model:** `src/memory/*`
  - Dexie persistence, concept mastery, learner modeling, and retrieval context assembly.
- **Architecture cognition layer:** `/brain/`
  - Generated knowledge graphs, runtime telemetry, debug artifacts, verification reports.

3) Core Runtime Layer (Application Orchestration)

3.1 Route & view orchestration

- **Routerless rendering:** `src/App.tsx` uses a Zustand `activeView` state instead of URL routing.
- Supported views: `study`, `analytics`, `revision`, `admin` (brain view is present but not an active route).
- **Lazy loading:** Analytics, Revision, and Admin routes are code-split with `React.lazy` and `Suspense`.

3.2 State management & persistence

- **Zustand store:** `src/store/index.ts` holds UI state (PDF, annotations, models, usage analytics, web search cache, etc.).
- **Persistence:** subsets of store state are persisted via `zustand/persist`.
- Usage analytics are recorded locally (chat, web, voice usage) with a storage key `usage_analytics_v1`.

3.3 Runtime telemetry and instrumentation

- **Runtime events:** `src/brain-runtime/runtimeTelemetry.ts` exposes `window.__BRAIN_RUNTIME__` with event capture.
- **Render profiling:** `BrainRenderProfiler` (React Profiler wrapper) emits `render` events.
- **Instrumentation hooks:**
  - `instrumentFetch` logs API calls (path, method, status, latency).
  - `instrumentWebSocket` logs socket lifecycle and message volume.
  - `instrumentZustand` logs state mutation fields.
  - `instrumentDexie` logs database reads/writes with timing and errors.
- **Runtime enablement:** via `VITE_BRAIN_RUNTIME=true` or `localStorage.brain_runtime=1`.

4) UI Layer (Views + Core Components)

4.1 Study View (primary workspace)

- **Location:** `src/views/StudyView.tsx`
- **Purpose:** orchestrates the PDF study surface + chat panel + landing storytelling UI.
- **Key behaviors:**
  - PDF upload (drag/drop + input), replace, and clear flows.
  - Preserves chat history with a single welcome reset on first load.
  - Coordinates PDF state (`pdfUrl` , `page` , `scale` , `totalPages`) through Zustand.

4.2 PDF Reader & annotation surface

- **Location:** `src/components/PdfViewer.tsx`
- **Stack:** `react-pdf` + PDF.js worker.
- **Capabilities:**
  - Zoom, fit-width, page navigation, keyboard shortcuts.
  - Text selection normalization and annotation types: highlight, underline, strikethrough, sticky notes.
  - Title extraction: captures page canvas, calls `POST /api/title` , updates learning book title.

4.3 Chat Panel

- **Location:** `src/components/ChatPanel.tsx`
- **Capabilities:**
  - Streaming SSE via `/api/chat` with status phases and tool call updates.
  - Rich output: Markdown, Mermaid, syntax highlighting (Shiki), code execution shell UI.
  - Integrated tools: flashcards, graph updates, web search citations, TTS playback, voice mode.
  - Hooks into memory orchestration to save interactions and update learning books.

4.4 Revision & Library

- **Location:** `src/views/RevisionView.tsx`
- **Purpose:** paper-style library for learning books, concepts, and active-recall flashcards.
- **Notable details:**
  - Flashcard review UX with spaced-repetition scoring.
  - Built-in Tutor System Architecture book seeded into Dexie (`tutorBook.json`).

4.5 Analytics

- **Location:** `src/views/AnalyticsView.tsx`
- **Purpose:** visualize mastery, confidence, session counts, and distributions.
- **Stack:** Recharts for bar/pie/area charts, data from Dexie.

4.6 Admin Diagnostics

- **Location:** `src/views/AdminView.tsx`
- **Purpose:** trace logs, server console stream, and debug run dashboards.
- **Sources:**
  - Trace logs from Dexie (`traceLogs`).
  - Server console via `ws://<host>/ws/debug` .
  - Debug run artifacts loaded from `/api/debug/*` endpoints.

4.7 Brain Map (3D graph)

- **Location:** `src/views/BrainView.tsx`
- **Purpose:** 3D force graph for learner, books, and concepts using `react-force-graph-3d` .
- **Memory safety:** cached geometries/materials are disposed on unmount to prevent GPU leaks.

5) Memory Layer (Persistence + Learner Model)

5.1 Persistent storage

- **Location:** `src/memory/longterm.memory.ts`
- **Database:** Dexie `NeuralNestBrain` (v6)
- **Tables:** concepts, misconceptions, sessions, interactions, flashcards, traceLogs, learningBooks, learningBookConcepts, learningEntries.

5.2 Memory orchestration

- **Location:** `src/memory/memory.orchestrator.ts`
- **Key responsibilities:**
  - Creates sessions and a per-session learning book.
  - Clears generated session library data (while preserving the built-in tutor book).
  - Logs interactions with embeddings and trace explanations (`/api/trace`).
  - Requests `/api/learning-book-update` to expand books, chapters, and concepts.
  - Writes `learningEntries` for each conversation and announces active book changes.

5.3 Embeddings and retrieval

- **Location:** `src/memory/memory.embeddings.ts`
- **Approach:** deterministic 384-dimensional hashed embeddings (no browser ML runtime).
- **Retrieval:** `MemoryOrchestrator.getRelevantContext()` scores interactions and concepts with cosine similarity and injects learner model directives.

5.4 Learner model pipeline

- **Location:** `src/memory/learner.model.ts` + related subsystems
- **Sub-modules:**
  - **BKT:** `bkt.engine.ts` updates P(L) per concept.
  - **ZPD:** `zpd.calculator.ts` computes independent / ZPD / not-yet zones.
  - **Scaffolding:** `scaffolding.engine.ts` maps mastery to support level.
  - **Misconceptions:** `misconception.graph.ts` tracks unresolved misunderstandings.
  - **Prerequisites:** `prerequisite.dag.ts` enforces prerequisite readiness.
  - **Cognitive load:** `cognitive.load.ts` classifies overload based on latency + retries.
  - **Illusion detection:** `illusion.detector.ts` checks confidence vs. performance.
  - **Productive failure:** `productive.failure.ts` classifies struggle state.
- **Output:** `getTutorInstructions()` injects pedagogically grounded directives into the chat system prompt.

6) Server Layer (Express + LLM/Voice/Web)

6.1 Core server capabilities

- **Location:** `server.ts`
- **Stack:** Express with SSE, WebSocket, OpenAI SDK (OpenRouter), Deepgram, Serper.
- **Run modes:**
  - Dev: Vite middleware.
  - Prod: static `dist` hosting plus Express APIs.

6.2 Key APIs

- **/api/chat (POST, SSE):**
  - Streams responses and tool calls; supports web search, flashcards, graph updates.
  - Injects memory context and custom system prompts.
  - Detects freshness queries and calls Serper search.
  - Emits usage metadata (tokens, cost, model).
- **/api/tts (GET):**
  - Deepgram TTS streaming with usage headers.

- **/api/title (POST):**
  - Uses Qwen vision to extract a short topic title from PDF page image.
- **/api/generate-persona (POST):**
  - Builds a system prompt from user persona description.
- **/api/trace (POST):**
  - Converts raw actions into a readable trace paragraph for Admin logs.
- **/api/learning-book-update (POST):**
  - Uses a learning-book agent to update session books, chapters, concepts.
  - Returns JSON schema; has a safe fallback path.
- **/api/generate-flashcards (POST):**
  - Generates flashcards using tool-call constraints.
- */api/debug/ (GET/POST):**
  - Orchestrates long-horizon debug runs and returns progress artifacts.

6.3 Web search normalization

- **Location:** `server/web-search.ts`
- **Serper integration:** normalizes organic/news/topStories rows, dedupes by canonical URL, assigns stable IDs.
- **Freshness detection:** regex-based triggers for news/current queries.

6.4 Voice agent proxy

- **WebSocket:** `/api/voice-agent`
- **Deepgram agent:** relays audio, sends agent config (listen/think/speak), streams output.
- **Usage:** tracks input/output audio seconds and emits usage events.

7) End-to-End Data Flow (User -> Tutor -> Memory)

1. **User uploads PDF** in Study View → `PdfViewer` loads, renders, and extracts page image.
2. **Title extraction** → `/api/title` returns a short topic title, updates session learning book title.
3. **User chats** → `ChatPanel` streams `/api/chat` SSE with tool calls (flashcards/graph/web).
4. **Memory update** → `MemoryOrchestrator` stores interactions, embeddings, and learning book entries.
5. **Revision view** pulls books, concepts, and flashcards from Dexie for review.
6. **Analytics view** charts mastery/interaction/session data from Dexie.
7. **Admin view** consumes trace logs (Dexie) and debug run artifacts (server) for audits.

8) Operational Boundaries & Notes

- **High-risk mutation boundaries:** Dexie schema, server API contracts, Zustand store shape, chat streaming format, and `/brain` generated artifacts.
- **Environment keys:** `OPENROUTER_API_KEY`, `DEEPGRAM_API_KEY`, `SERPER_API_KEY` (with UI overrides).
- **Build & lint:** `npm run lint`, `npm run build`.

Output: This audit document is intended to be the human-readable summary for stakeholders, onboarding, and future architectural reviews. It complements (but does not replace) the `/brain` artifacts and the existing `TUTOR_ARCHITECTURE.md` book.